


# Device Model

## M2 SETI-B4 Embedded Linux

Maria Mushtaq
Maria.mushtaq@telecom-paris.fr
2022-2023



Institut Mines-Télécom

# Plan

- Introduction
- Example of a USB device driver
- Static Discovery of devices
- ZOOM on I2C devices
- I2C communication API

# Presentation

- The kernel manages, through drivers, many different devices on many different hardware platforms
- Some functions are common to all drivers
    - Declaration of the list of supported hardware devices
    - Call by the kernel when such a device is detected...

# Model

- A physical device is connected to the processor via a bus

  - Example: a mass storage device connected to the USB bus

  - In the absence of an explicit bus (for example for a device directly mapped in memory) there is a generic bus, the platform bus

- This bus is often managed by a controller (controller or adapter) which is itself a device connected to another bus

  - For example, a USB bus is managed by a controller (OHCI, UHCI, EHCI...) of the USB bus, which is itself a device connected to a PCI bus

# Model

- A physical device is managed by a driver

  - A single driver can manage several physical devices (for example several accelerometers of the same model, or accelerometers of similar models)

- Several devices providing the same services can be grouped into classes

  - For example the sound class which groups all the audio devices

# Data Structures
# #include <linux/device.h>

- struct bus_type: represents a bus
- struct device_driver: represents a device driver
- struct device: represents a physical device
- struct class: represents a class of devices
- These structures can be specialized according to the bus type

  - USB device: struct usb_device (which contains a struct device field)

  - USB device driver: struct usb_device_driver (which contains a field of type device_driver)

# Basic architecture of a driver

- Initialization function

  - Declaration of supported physical devices - Registration to the corresponding bus

- Probe function

  - Called by the kernel when a supported device is detected

- Disconnect (or remove) function

  - Called by the kernel when a supported device is disconnected

- Cleaning function

  - Unregister from the bus

# Plan

- Introduction
- Example of a USB device driver
- Static Discovery of devices
- ZOOM on I2C devices
- I2C communication API

# An example of a USB device driver

- Study of a simple USB device driver: LED

  /drivers/usb/misc/usbled.c

- In version 4.7 of the kernel (has since been moved to the HID module)
- We will only focus here on the part of the driver managing the hardware part (we will ignore for the moment the interface with the user space)

# Usbled.c (simplified)

```
/* headers */
static const struct usb_device_id id_table[] = {
•{  USB_DEVICE(0x0fc5, 0x1223),
.driver_info = DELCOM_VISUAL_SIGNAL_INDICATOR },
•{  USB_DEVICE(0x1d34, 0x0004),
.driver_info = DREAM_CHEEKY_WEBMAIL_NOTIFIER },
/* ... */
{ },
};
MODULE_DEVICE_TABLE(usb, id_table);
static int led_probe(struct usb_interface *interface ,
const struct usb_device_id *id) { /* ... */ }
static void led_disconnect(struct usb_interface *interface) { /* ... */ } static struct usb_driver led_driver = {


.name = .probe = .disconnect = .id_table =
"usbld", led_probe , led_disconnect , id_table ,
}; module_usb_driver(led_driver);
MODULE_AUTHOR(DRIVER_AUTHOR); MODULE_DESCRIPTION(DRIVER_DESC); MODULE_LICENSE("GPL");
```

# First Observation

```
/* ... */ module_usb_driver(led_driver);
MODULE_AUTHOR(DRIVER_AUTHOR);
MODULE_DESCRIPTION(DRIVER_DESC);
MODULE_LICENSE("GPL");
```

- The MODULE_AUTHOR, MODULE_DESCRIPTION and MODULE_LICENSE macros are present
- But where are the initialization and cleaning functions?
- They are hidden under the macro module_usb_driver

# Initialization and cleaning functions

- The macro module_usb_driver generates the following code

```
static int __init led_driver_init(void)
{
        return usb_register(&(led_driver));
}
module_init(led_driver_init);

static void __exit led_driver_exit(void)
{
        usb_deregister(&(led_driver));
}
module_exit(led_driver_exit);
```

- In a driver, in general, the initialization and cleaning functions simply register the driver with the appropriate subsystem (here USB)
- Hence the practical use of a macro to generate them automatically

# Driver Registration

```
static struct usb_driver led_driver = {
        .name =         "usbled",
        .probe =        led_probe,
        .disconnect =   led_disconnect,
        .id_table =     id_table,
};
```

- name : name of the driver
- probe : pointer to the function to call when a physical device that the driver can manage is detected
- disconnect : pointer to the function to call when a physical device managed by the driver is disconnected
- id_table : table allowing to identify the USB devices that our driver is able to manage

# Identification of managed devices

```
static const struct usb_device_id id_table[] = {
        { USB_DEVICE(0x0fc5, 0x1223),
          .driver_info = DELCOM_VISUAL_SIGNAL_INDICATOR },
        { USB_DEVICE(0x1d34, 0x0004),
          .driver_info = DREAM_CHEEKY_WEBMAIL_NOTIFIER },
        /* ... */
        { },
};
MODULE_DEVICE_TABLE(usb, id_table);
```

- id_table lists the devices that the driver can handle
- For example, the driver announces that it knows how to manage a USB device whose vendor id is 0x0fc5 and product id is 0x1223
- So if such a device is detected on the bus, the kernel will inform the driver (by calling the led_probe function)

# Identification of managed devices

```
static const struct usb_device_id id_table[] = {
        { USB_DEVICE(0x0fc5, 0x1223),
          .driver_info = DELCOM_VISUAL_SIGNAL_INDICATOR },
        { USB_DEVICE(0x1d34, 0x0004),
          .driver_info = DREAM_CHEEKY_WEBMAIL_NOTIFIER },
        /* ... */
        { },
};
MODULE_DEVICE_TABLE(usb, id_table);
```

- The driver_info field allows our driver, when detecting a device, to know which table entry corresponds to it

  - Useful for example if each device has to be managed differently

# Identification of managed devices

```
static const struct usb_device_id id_table[] = {
        { USB_DEVICE(0x0fc5, 0x1223),
          .driver_info = DELCOM_VISUAL_SIGNAL_INDICATOR },
        { USB_DEVICE(0x1d34, 0x0004),
          .driver_info = DREAM_CHEEKY_WEBMAIL_NOTIFIER },
        /* ... */
        { },
};
MODULE_DEVICE_TABLE(usb, id_table);
```

- The MODULE_DEVICE_TABLE macro allows to export this information to the user space (via the depmod tool and the /lib/modules/xxx/modules.alias file)
- This file is used by udev to automatically load the appropriate module when a device is detected

# Detection of a managed device

```
static  int led_probe(struct usb_interface *interface ,
const struct usb_device_id *id) { /* ... */ }
```

- This function has several objectives

  - Verify that the driver can effectively manage the device

  - Initialize the device and the data structures

- necessary to manage it (interrupt handler...)

  - Set up the communication with the user space (registration with a particular framework...)

- It returns 0 if everything went well or a negative error code

- It can be called several times if several different devices (for example, two USB LEDs) are present

# Deconnecting a managed device

```
static void led_disconnect(struct usb_interface *interface) { /* ... */ }
```

- When a device managed by our driver is disconnected, the kernel will call the function declared in the disconnect field of the usb_driver structure of the corresponding driver, here led_disconnect
- This function will have to free the resources allocated for this device

# To conclude

- The driver of a USB device is composed of three important parts

  - A table to declare the devices it can handle
  - A probe function called by the kernel when a physical device is detected
  - A disconnect function called by the kernel when a physical device is disconnected

- The physical device/driver association is simple because the USB bus allows to identify which devices are connected

# Plan

- Introduction
- Example of a USB device driver
- Static Discovery of devices
- ZOOM on I2C devices
- I2C communication API

# Problem Statement

- Some buses do not offer the possibility to identify which components are connected to them

  - Examples : SPI bus, I2C bus...

- How can the kernel know that a peripheral is connected?
- How can the kernel make the association between a device and the corresponding driver?

# Solutions

- The kernel cannot obtain this information alone
- Two mechanisms exist to provide it with this information
  - Hard description of the platform in the kernel
  - Device Tree

# Example

- Seiko Instruments S-35390A
- I2C device Real time clock (RTC)
- /drivers/rtc/rtc-s35390a.c, version 4.18

# Rtc-s35390a.c (simplified)

```
static const struct i2c_device_id s35390a_id[] = {
        { "s35390a", 0 },
        { }
};
MODULE_DEVICE_TABLE(i2c, s35390a_id);

static const struct of_device_id s35390a_of_match[] = {
        { .compatible = "s35390a" },
        { .compatible = "sii,s35390a" },
        { }
};
MODULE_DEVICE_TABLE(of, s35390a_of_match);

static int s35390a_probe(struct i2c_client *client,
                         const struct i2c_device_id *id) { /* ... */ }
static int s35390a_remove(struct i2c_client *client) { /* ... */ }

static struct i2c_driver s35390a_driver = {
        .driver         = {
                .name   = "rtc-s35390a",
                .of_match_table = of_match_ptr(s35390a_of_match),
        },
        .probe          = s35390a_probe,
        .remove         = s35390a_remove,
        .id_table       = s35390a_id,
};
module_i2c_driver(s35390a_driver);
```

# Rtc-s35390a.c (simplified)

```
static int s35390a_probe(struct i2c_client *client,
                         const struct i2c_device_id *id) { /* ... */ }
static int s35390a_remove(struct i2c_client *client) { /* ... */ }

static struct i2c_driver s35390a_driver = {
        .driver         = {
                .name   = "rtc-s35390a",
                .of_match_table = of_match_ptr(s35390a_of_match),
        },
        .probe          = s35390a_probe,
        .remove         = s35390a_remove,
        .id_table       = s35390a_id,
};
module_i2c_driver(s35390a_driver);
```

We find :

- The macro to create the initialization and cleaning functions
- The structure representing the driver
- The probe and remove functions

# Rtc-s35390a.c (simplified)

```
static const struct i2c_device_id s35390a_id[] = {
        { "s35390a", 0 },
        { }
};
MODULE_DEVICE_TABLE(i2c, s35390a_id);

static const struct of_device_id s35390a_of_match[] = {
        { .compatible = "s35390a" },
        { .compatible = "sii,s35390a" },
        { }
};
MODULE_DEVICE_TABLE(of, s35390a_of_match);

static struct i2c_driver s35390a\_driver = {
        .driver         = {
                .name   = "rtc-s35390a",
                .of_match_table = of_match_ptr(s35390a_of_match),
        },
        .probe          = s35390a_probe,
        .remove         = s35390a_remove,
        .id_table       = s35390a_id,
};
```

These two structures correspond to the two mechanisms described above for the identification of devices

# Static Declaration

- Source file describing programmatically the devices present and their characteristics on a platform
- The platform is chosen during the kernel configuration (or identified by the bootloader which then provides the information to the kernel)

## Static Declaration
## /arch/arm/mach-imx/mach-armadillo5x0.c

```
static struct i2c_board_info armadillo5x0_i2c_rtc = {
        I2C_BOARD_INFO("s35390a", 0x30),
};

static void __init armadillo5x0_late(void)
{
        /* ... */
        i2c_register_board_info(1, &armadillo5x0_i2c_rtc, 1);
        /* ... */
}

MACHINE_START(ARMADILLO5X0, "Armadillo-500")
        /* ... */
        .init_late      = armadillo5x0_late,
MACHINE_END
```

# Static Declaration
# /arch/arm/mach-imx/mach-armadillo5x0.c

```
static struct i2c_board_info armadillo5x0_i2c_rtc = {
        I2C_BOARD_INFO("s35390a", 0x30),
};

static void __init armadillo5x0_late(void)
{
        /* ... */
        i2c_register_board_info(1, &armadillo5x0_i2c_rtc, 1);
        /* ... */
}

MACHINE_START(ARMADILLO5X0, "Armadillo-500")
        /* ... */
        .init_late      = armadillo5x0_late,
MACHINE_END
```

# Static Declaration
## Association

- /drivers/rtc/rtc-s35390a.c

```
static const struct i2c_device_id s35390a_id[] = {
        { "s35390a", 0 },
        { }
};
MODULE_DEVICE_TABLE(i2c, s35390a_id);

static struct i2c_driver s35390a_driver = {
        .id_table       = s35390a_id,
        /* ... */
};
```

- /arch/arm/mach-imx/mach-armadillo5x0.c

```
static struct i2c_board_info armadillo5x0_i2c_rtc = {
        I2C_BOARD_INFO("s35390a", 0x30),
};
```

# Static Declaration Inconvenience

- The platform configuration is hard-coded in the kernel
- Need to recompile the kernel if the peripherals are modified
- Lack of flexibility

# Device tree
# Introduction

- Tree-like data structure describing the hardware on a platform
- Specifications developed by the Open Firmware project and derived from SPARC-based machines
- Source file (Device Tree Source, DTS) "compiled in binary form (Device Tree Blob, DTB) by the dtc (Device Tree Compiler) tool provided by the kernel
- The DTB is provided to the kernel at boot time

## Device tree
## /arch/arm/boot/dts/r8a7740-armadillo800eva.dts

```
/dts-v1/;
/* #include ... */
&i2c2 {
}; };
status = "okay"; rtc@30 {
compatible = "sii,s35390a";
reg = <0x30>;
```

## Device tree
## /arch/arm/boot/dts/r8a7740-armadillo800eva.dts

```
/dts-v1/;
/* #include ... */
&i2c {
}; };
status = "okay"; rtc@30 {
compatible = "sii,s35390a";
 reg = <0x30>;
```

- Refers to a node defined elsewhere in the tree

## Device tree
## /arch/arm/boot/dts/r8a7740-armadillo800eva.dts

```
/dts-v1/;
/* #include ... */
&i2c {
rtc@30 {
}; };
status = "okay";
compatible = "sii,s35390a";
reg = <0x30>;
```

- We add a child node to this node

## Device tree
## /arch/arm/boot/dts/r8a7740-armadillo800eva.dts

```
/dts-v1/;
/* #include ... */
&i2c {
}; };
status = "okay"; rtc@30 {
compatible = "sii,s35390a";
reg = <0x30>;
```

- The compatible property will allow to make the relation between a device and the driver able to manage it

## Device tree
## /arch/arm/boot/dts/r8a7740-armadillo800eva.dts

```
//dts-v1/;
/* #include ... */
&i2c {
}; };
status = "okay"; rtc@30 {
compatible = "sii,s35390a";
reg = <0x30>;
};};
```

- The reg property will indicate the device address

# Device tree
# /arch/arm/boot/dts/r8a7740-armadillo800eva.dts

- /drivers/rtc/rtc-s35390a.c

```
static const struct of_device_id s35390a_of_match[] = {
        { .compatible = "s35390a" },
        { .compatible = "sii,s35390a" },
        { }
};
MODULE_DEVICE_TABLE(of, s35390a_of_match);

static struct i2c_driver s35390a_driver = {
        .driver         = {
                .name   = "rtc-s35390a",
                .of_match_table = of_match_ptr(s35390a_of_match),
        },
        /* ... */
};
```

## /arch/arm/boot/dts/r8a7740-armadillo800eva.dts

```
 &i2c2 {
         rtc@30 {
                 compatible = "sii,s35390a";
                 reg = <0x30>;
         };
 };
```

# Device Tree Advantages

- Modifiable without recompiling the kernel
- Same kernel image for several platforms
- Dynamically modifiable (overlay mechanism)
- Mandatory for ARM-based systems-on-chip, usable on other architectures (x86, SPARC, PowerPC...)

# rtc-s35390a.c (Simplified)
# Two Mechanisms

```
static const struct i2c_device_id s35390a_id[] = {
        { "s35390a", 0 },
        { }
};
MODULE_DEVICE_TABLE(i2c, s35390a_id);

static const struct of_device_id s35390a_of_match[] = {
        { .compatible = "s35390a" },
        { .compatible = "sii,s35390a" },
        { }
};
MODULE_DEVICE_TABLE(of, s35390a_of_match);

static struct i2c_driver s35390a_driver = {
        .driver         = {
                .name   = "rtc-s35390a",
                .of_match_table = of_match_ptr(s35390a_of_match),
        },
        .probe          = s35390a_probe,
        .remove         = s35390a_remove,
        .id_table       = s35390a_id,
};
module_i2c_driver(s35390a_driver);
```

# rtc-s35390a.c (Simplified)
# Two Mechanisms

```
static const struct i2c_device_id s35390a_id[] = {
        { "s35390a", 0 },
        { }
};
MODULE_DEVICE_TABLE(i2c, s35390a_id);

static const struct of_device_id s35390a_of_match[] = {
        { .compatible = "s35390a" },
        { .compatible = "sii,s35390a" },
        { }
};
MODULE_DEVICE_TABLE(of, s35390a_of_match);

static struct i2c_driver s35390a_driver = {
        .driver         = {
                .name   = "rtc-s35390a",
                .of_match_table = of_match_ptr(s35390a_of_match),
        },
        .probe          = s35390a_probe,
        .remove         = s35390a_remove,
        .id_table       = s35390a_id,
};
module_i2c_driver(s35390a_driver);
```

# Summary

- Two mechanisms allow the association between a physical device present on a machine and the corresponding driver, if the discovery of devices cannot be automated
  - Programmatic declaration in the kernel sources specific to a platform
  - Device tree
- For both, comparison of characters
- Both mechanisms are often present in the driver code

# Plan

- Introduction
- Example of a USB device driver
- Static Discovery of devices
- ZOOM on I2C devices
- I2C communication API

# Bus I2C
# Presentation

- Inter-Integrated Circuit
- Designed in 1982 by Philips Semiconductor
- Used to connect, over short distances and at low speed, peripherals to a microprocessor or microcontroller

    - Examples: temperature sensor, accelerometer, flash memory...

# Skeleton of an I2C device driver Example

- Seiko Instruments S-35390A I2C device
- Real time clock (RTC)
- /drivers/rtc/rtc-s35390a.c, version 4.18

# rtc-s35390a.c (simplified)

```
static const struct i2c_device_id s35390a_id[] = {
        { "s35390a", 0 },
        { }
};
MODULE_DEVICE_TABLE(i2c, s35390a_id);

static const struct of_device_id s35390a_of_match[] = {
        { .compatible = "s35390a" },
        { .compatible = "sii,s35390a" },
        { }
};
MODULE_DEVICE_TABLE(of, s35390a_of_match);

static int s35390a_probe(struct i2c_client *client,
                         const struct i2c_device_id *id) { /* ... */ }
static int s35390a_remove(struct i2c_client *client) { /* ... */ }

static struct i2c_driver s35390a_driver = {
        .driver         = {
                .name   = "rtc-s35390a",
                .of_match_table = of_match_ptr(s35390a_of_match),
        },
        .probe          = s35390a_probe,
        .remove         = s35390a_remove,
        .id_table       = s35390a_id,
};
module_i2c_driver(s35390a_driver);
```

# rtc-s35390a.c (simplified)
# Static Association

```
static const struct i2c_device_id s35390a_id[] = {
        { "s35390a", 0 },
        { }
};
MODULE_DEVICE_TABLE(i2c, s35390a_id);

static struct i2c_driver s35390a_driver = {
        .id_table       = s35390a_id,
        /* ... */
};
```

- This structure is used for static association

  - Each entry contains two fields: a string and an integer

  - The last entry must be empty

  - It is associated to the driver thanks to the field id_table of the i2c_driver structure

  - The MODULE_DEVICE_TABLE macro allows to export this information to the user space

# rtc-s35390a.c (simplified)
# Association with a Device Tree

```
static const struct of_device_id s35390a_of_match[] = {
        { .compatible = "s35390a" },
        { .compatible = "sii,s35390a" },
        { }
};
MODULE_DEVICE_TABLE(of, s35390a_of_match);

static struct i2c_driver s35390a_driver = {
        .driver         = {
                .of_match_table = of_match_ptr(s35390a_of_match), /* ... */
        }, /* ... */
};
```

- This structure is used for the association using a tree of devices

  - Each entry contains a compatible field

  - The last entry must be empty

  - It is associated to the driver thanks to the field of_match_table of the i2c_driver structure

  - The MODULE_DEVICE_TABLE macro allows to export this information to the user space

# rtc-s35390a.c (simplified)
# Probe Function

```
static int s35390a_probe(struct i2c_client *client,
                         const struct i2c_device_id *id) { /* ... */ }

static struct i2c_driver s35390a_driver = {
        .probe          = s35390a_probe,
        /* ... */
};
```

- Called when a supported device is detected
- Arguments

  - client : represents the I2C device

  - id : pointer to the entry of the array s35390a_id corresponding to the device (NULL if the device tree has been used)

- Returns 0 if all goes well, a negative number in case of error

# rtc-s35390a.c (simplified)
# Probe Function

- It is possible to retrieve private information
  - Driver_data field of the i2c_device_id structure
  - Data field of the of_device_id structure

```
if (client->dev.of_node) {
    /* Device tree */
    const struct of_device_id *of_id =
        of_match_device(s35390a_of_match, &client->dev);
    variant = of_id->data;      /* Champ data de of_device_id */
} else {
    /* Association statique */
    variant = id->driver_data;  /* Champ driver_data de i2c_device_id */
}
```

# rtc-s35390a.c (simplified)
# Remove Function

```
static int s35390a_remove(struct i2c_client *client) { /* ... */ }

static struct i2c_driver s35390a_driver = {
        .remove         = s35390a_remove,
        /* ... */
};
```

- Called when the device is "disconnected

  - Unloading the module

  - Modification of the tree of devices

# rtc-s35390a.c (simplified)
# Driver Declaration

```
static struct i2c_driver s35390a_driver = {
        .driver         = {
                .name   = "rtc-s35390a",
                .of_match_table = of_match_ptr(s35390a_of_match),
        },
        .probe          = s35390a_probe,
        .remove         = s35390a_remove,
        .id_table       = s35390a_id,
};
module_i2c_driver(s35390a_driver);
```

- name : name of the driver (the same as the name of the module)
- The module_i2c_driver macro automatically creates the initialization and cleaning functions

# Plan

- Introduction
- Example of a USB device driver
- Static Discovery of devices
- ZOOM on I2C devices
- I2C communication API

# The I2C bus
# Transmissions

- Always initiated by a master 7-bit addresses
- Start of an exchange

  - Master: START + 7 address bits + R(1)/W(0) - Slave (if present) : ACK

- If writing (master to slave)

  - Master: one data byte

  - Slave: ACK or NACK...

- If reading (slave to master)

  - Slave: one data byte

  - Master : ACK (continue) or NACK (end)...

- Master: STOP

# Basic API master
# Write (master to slave)

```
int i2c_master_send(const struct i2c_client *client ,
const char *buf, int count);
```

- client : structure representing the slave device
- buf : pointer to the buffer containing the data to be transmitted to the slave
- count : number of bytes to transmit
- Returns the number of bytes transmitted or a negative error code

# Basic API master
# Read (slave to master)

```
int i2c_master_recv(const struct i2c_client *client ,
 char *buf, int count);
```

- client : structure representing the slave device
- buf : pointer to the buffer containing the data to be transmitted to the slave
- count : number of bytes to transmit
- Returns the number of bytes transmitted or a negative error code

# Basic API master
# Message Transfer

```
int i2c_transfer(struct i2c_adapter *adap, struct i2c_msg
*msg int num );
```

- adap : structure representing the I2C controller (can be retrieved from the client : client->adapter)
- msgs : array of messages num : number of messages
- Returns the number of messages transferred or a negative error code

# Basic API master
# Message Transfer

```
struct i2c_msg {
        __u16 addr;         /* slave address */
        __u16 flags;
        __u16 len;          /* msg length */
        __u8 *buf;          /* pointer to msg data */
};
#define I2C_M_RD 0x0001     /* read data, from slave to master
```

- addr : slave address (can be retrieved at from the client : client->addr)
- flags : flags
- len : number of data to read or write
- buf : pointer to the buffer containing the data to be written or, in case of writing, where the read data will be stored

# Your Turn to Play

- You will write the beginning of a device driver for an I2C accelerometer
- This device is emulated by QEMU